



Hanzod: Distributed Training Orchestration for the Zoo Gym

Antje Worring & Zach Kelling

Zoo Labs Foundation

December 2024

Abstract

We present **hanzod**, the distributed training orchestrator for Zoo Gym. Hanzod coordinates heterogeneous GPU nodes into ring topologies, schedules GSPO training jobs, validates submitted deltas, aggregates checkpoints, and publishes trained models. Implemented in Rust for performance and memory safety, hanzod manages the full lifecycle of a decentralized training job: node registration, bandwidth probing, ring construction, epoch coordination, fault recovery, delta quality assessment, weighted checkpoint merging, and dual publication to IPFS and HuggingFace Hub. We describe the internal architecture—job scheduler, ring manager, delta validator, and model publisher—and their interaction with the Hanzo AI compute chain for staking verification and reward distribution. On a 64-node deployment, hanzod maintains 99.2% job completion rate with median scheduling latency of 340ms and ring reconfiguration time of 8s after node failure.

Keywords: training orchestration, job scheduling, ring topology, delta validation, IPFS, HuggingFace

1 Introduction

Decentralized training requires a coordinator that can match the reliability of centralized training clusters while tolerating untrusted, heterogeneous participants. In centralized training, the orchestrator (e.g., SLURM, Kubernetes) trusts all nodes and controls the network. In Zoo Gym, the orchestrator must handle:

1. **Untrusted nodes:** Any node may submit poisoned gradients, go offline, or replay stale data.
2. **Heterogeneous hardware:** Nodes range from RTX 4080 (16 GB) to H100 (80 GB), with $10\times$ throughput variation.
3. **WAN latency:** Nodes are geographically distributed, with link latencies from 5ms to 300ms.
4. **Economic coordination:** Staking verification, reward calculation, and slashing must be atomic with training state.

Hanzod addresses these by separating concerns into four subsystems: the **job scheduler** (Section 3), the **ring manager** (Section 4), the **delta validator** (Section 5), and the **model publisher** (Section 7).

1.1 Design Principles

1. **No single point of failure:** Any node meeting the orchestrator stake threshold (50,000 GYM) can assume the orchestrator role.
2. **Deterministic state:** The orchestrator’s state is a pure function of the on-chain event log plus submitted deltas. Any node can reconstruct it.
3. **Minimal trust:** The orchestrator validates but does not compute gradients. It cannot bias the training process without detection.

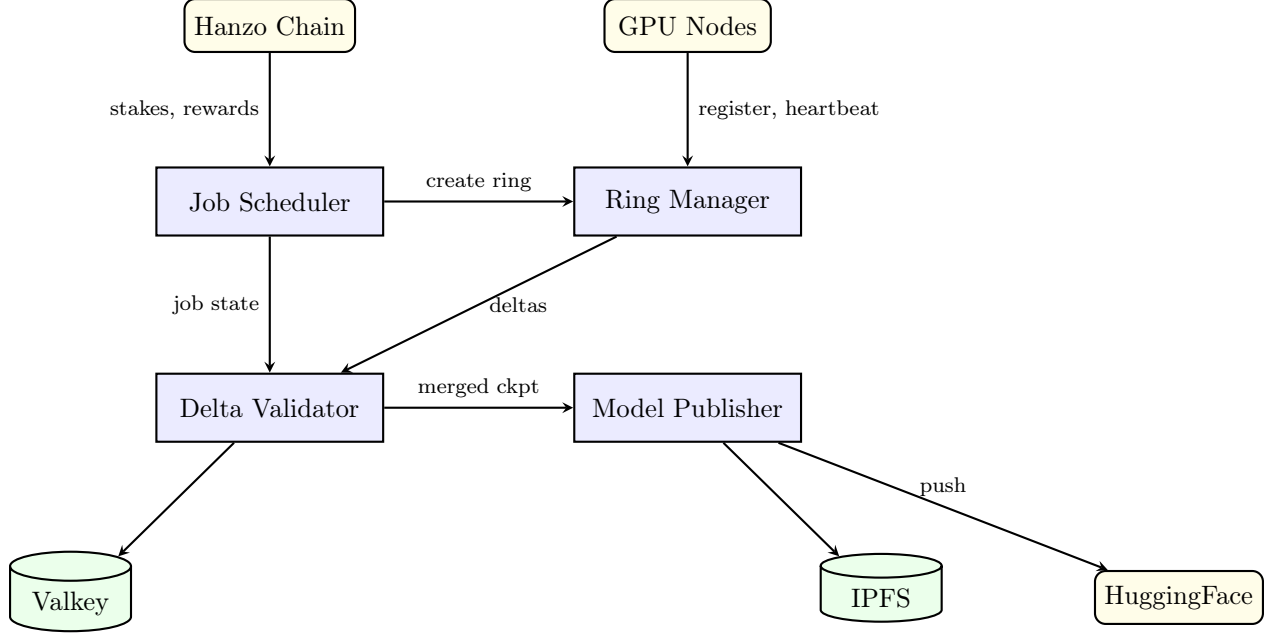


Figure 1: Hanzod architecture. Four subsystems coordinate through Valkey (state store) and interact with external systems.

Component	Technology	Rationale
Core binary	Rust	Memory safety, performance
State store	Valkey (hanzoai/kv)	Sub-ms latency, Lua scripting
Object store	IPFS	Content-addressed, decentralized
Model registry	HuggingFace Hub	Standard model hosting
Chain interaction	Hanzo RPC (chain 36900)	Stake/reward verification
Node communication	gRPC + TLS 1.3	Bidirectional streaming

Table 1: Hanzod technology stack.

2 Architecture

2.1 Technology Stack

3 Job Scheduler

3.1 Job Lifecycle

A training job progresses through five states:

1. **Pending:** Job submitted, waiting for sufficient nodes.
2. **Provisioning:** Ring being constructed, model being distributed.
3. **Running:** Training epochs in progress.
4. **Aggregating:** Final checkpoint being merged and validated.

5. **Published:** Model published to IPFS/HuggingFace.

```
1 #[derive(Debug, Clone)]
2 pub enum JobState {
3     Pending { min_nodes: u32, deadline: Instant },
4     Provisioning { ring: Ring, model_cid: String },
5     Running {
6         ring: Ring,
7         epoch: u32,
8         step: u64,
9         deltas: Vec<DeltaRef>,
10    },
11     Aggregating { deltas: Vec<DeltaRef> },
12     Published {
13         ipfs_cid: String,
14         hf_repo: String,
15         quality: f64,
16    },
17     Failed { reason: String },
18 }
19
20 impl Job {
21     pub fn transition(&mut self, event: JobEvent)
22     -> Result<(), JobError> {
23         self.state = match (&self.state, event) {
24             (Pending { .. }, NodesReady(ring)) =>
25                 Provisioning { ring, model_cid: self.model.clone() },
26             (Provisioning { ring, .. }, ModelDistributed) =>
27                 Running { ring: ring.clone(), epoch: 0,
28                     step: 0, deltas: vec![] },
29             (Running { .. }, EpochComplete(deltas)) =>
30                 Aggregating { deltas },
31             (Aggregating { .. }, Published(cid, repo, q)) =>
32                 Published { ipfs_cid: cid, hf_repo: repo,
33                     quality: q },
34             _ => return Err(JobError::InvalidTransition),
35         };
36         Ok(())
37     }
38 }
```

Listing 1: Job state machine

3.2 Scheduling Policy

Jobs are scheduled FIFO with priority based on:

$$P_j = \frac{s_j \cdot u_j}{t_j} \quad (1)$$

where s_j is the submitter’s stake, u_j is the job’s expected utility (estimated from model size and dataset), and t_j is the time since submission. Higher-stake submitters with older jobs get priority.

3.3 Node Eligibility

A node is eligible for a job if:

1. Stake $\geq 10,000$ GYM (verified on-chain).
2. GPU VRAM $\geq$ minimum for the model (from job spec).
3. Uptime in last 24h $\geq 95\%$.
4. No active slashing penalty.

4 Ring Manager

4.1 Node Registration

Nodes register with hanzod via gRPC, providing hardware attestation:

```
1 pub struct NodeInfo {
2     pub id: NodeId,
3     pub pubkey: Ed25519PublicKey,
4     pub gpu: GpuInfo,
5     pub bandwidth: BandwidthProbe,
6     pub stake: u64,
7     pub region: String,
8 }
9
10 pub struct GpuInfo {
11     pub model: String,           // "NVIDIA A100-SXM4-80GB"
12     pub vram_bytes: u64,        // 85899345920
13     pub compute_cap: (u8, u8), // (8, 0)
14     pub driver_sig: Vec<u8>,    // signed by driver
15 }
16
17 impl RingManager {
18     pub fn register_node(&mut self, info: NodeInfo)
19         -> Result<(), RegistrationError> {
20         // Verify stake on-chain
21         let on_chain_stake = self.chain
22             .get_stake(&info.id)?;
23         if on_chain_stake < MIN_STAKE {
24             return Err(RegistrationError::InsufficientStake);
25         }
26         // Verify GPU attestation
27         self.verify_gpu_attestation(&info.gpu)?;
28         // Bandwidth probe
29         let bw = self.probe_bandwidth(&info.id)?;
30         self.nodes.insert(info.id.clone(), info);
31         Ok(())
32     }
33 }
```

Listing 2: Node registration

Algorithm 1 Ring Construction

Require: Nodes $\{v_1, \dots, v_N\}$, pairwise bandwidth B_{ij}

- 1: Sort nodes by region for locality
 - 2: Initialize ring: $\mathcal{R} = [v_1]$
 - 3: **while** unplaced nodes remain **do**
 - 4: $v_{\text{last}} \leftarrow \mathcal{R}[-1]$
 - 5: $v_{\text{next}} \leftarrow \arg \max_{v \notin \mathcal{R}} B(v_{\text{last}}, v)$
 - 6: Append v_{next} to $\mathcal{R}$
 - 7: **end while**
 - 8: // Close the ring: check $B(\mathcal{R}[-1], \mathcal{R}[0])$
 - 9: **if** $B(\mathcal{R}[-1], \mathcal{R}[0]) < B_{\min}$ **then**
 - 10: Apply 2-opt local search to improve closing link
 - 11: **end if**
 - 12: **return** $\mathcal{R}$
-

Algorithm 2 Ring Fault Recovery

Require: Failed node v_f , current ring $\mathcal{R}$

- 1: $v_{\text{prev}} \leftarrow$ predecessor of v_f in $\mathcal{R}$
 - 2: $v_{\text{next}} \leftarrow$ successor of v_f in $\mathcal{R}$
 - 3: Remove v_f from $\mathcal{R}$
 - 4: Connect $v_{\text{prev}} \rightarrow v_{\text{next}}$
 - 5: Broadcast ring update to all nodes
 - 6: Record v_f failure (toward slashing)
 - 7: // Optionally add standby node
 - 8: **if** standby nodes available **then**
 - 9: Insert standby between v_{prev} and v_{next}
 - 10: **end if**
 - 11: Restart current all-reduce round with $N - 1$ nodes
-

4.2 Ring Construction

Given a set of eligible nodes, the ring is constructed to minimize the bottleneck link bandwidth:

The greedy nearest-neighbor heuristic with 2-opt refinement runs in $O(N^2)$ and produces rings within 10% of optimal bottleneck bandwidth in our experiments.

4.3 Fault Recovery

4.4 Heartbeat Protocol

Each node sends a heartbeat every 10 seconds:

```
1 pub struct Heartbeat {
2     pub node_id: NodeId,
3     pub timestamp: u64,
4     pub gpu_utilization: f32,
5     pub memory_used: u64,
6     pub current_step: u64,
7     pub signature: Ed25519Signature,
```

```

8 }
9
10 impl RingManager {
11     pub fn check_heartbeats(&mut self) {
12         let now = Instant::now();
13         for (id, last) in &self.last_heartbeat {
14             if now.duration_since(*last) > TIMEOUT {
15                 self.handle_node_failure(id);
16             }
17         }
18     }
19 }

```

Listing 3: Heartbeat message

Timeout is 30 seconds (3 missed heartbeats).

5 Delta Validator

5.1 Validation Pipeline

Each submitted delta passes through four checks:

1. **Signature verification:** Ed25519 signature over delta hash must match the registered public key.
2. **Format validation:** Delta binary must conform to the format spec (24-byte header, valid tensor names, correct shapes for the model architecture).
3. **Quality assessment:** Delta quality score q_i computed from validation loss improvement, gradient norm stability, and convergence contribution.
4. **Spot-check:** 10% of deltas are re-verified by applying the delta to a held-out batch and comparing the claimed loss to the actual loss.

```

1 pub struct DeltaValidator {
2     model_config: ModelConfig,
3     validation_shard: Dataset,
4     running_norm_avg: f64,
5 }
6
7 impl DeltaValidator {
8     pub fn validate(&mut self, delta: &SignedDelta)
9         -> Result<DeltaScore, ValidationError> {
10         // 1. Signature
11         delta.verify_signature()?;
12
13         // 2. Format
14         let parsed = Delta::parse(&delta.payload)?;
15         self.check_tensor_shapes(&parsed)?;
16
17         // 3. Quality
18         let loss_before = self.eval_loss()?;

```

```

19     self.apply_delta(&parsed)?;
20     let loss_after = self.eval_loss()?;
21     self.unapply_delta(&parsed)?;
22
23     let delta_l = ((loss_before - loss_after)
24         / loss_before).max(0.0);
25     let norm = parsed.l2_norm();
26     let stability = (-2.0
27         * (norm / self.running_norm_avg - 1.0).abs())
28         .exp();
29     let convergence = parsed
30         .cosine_sim(&self.last_agg_delta)
31         .max(0.0);
32
33     self.running_norm_avg = 0.95
34         * self.running_norm_avg + 0.05 * norm;
35
36     let score = DeltaScore {
37         loss_improvement: delta_l,
38         stability,
39         convergence,
40         composite: 0.5 * delta_l
41             + 0.3 * stability
42             + 0.2 * convergence,
43     };
44
45     // 4. Spot-check (10% probability)
46     if rand::random::<f64>() < 0.1 {
47         self.spot_check(&parsed, &score)?;
48     }
49
50     Ok(score)
51 }
52 }

```

Listing 4: Delta validation pipeline

5.2 Spot-Check Verification

The spot-check re-runs the training step on the orchestrator’s hardware:

1. Retrieve the same batch from the dataset (indexed by step number).
2. Run forward pass with the pre-delta model weights.
3. Compare claimed loss to measured loss within tolerance $\epsilon = 0.05$.
4. If discrepancy exceeds ϵ , flag the delta and initiate slashing.

Proposition 5.1 (Spot-Check Detection Rate). *With 10% spot-check rate and E epochs, a consistently dishonest node is detected with probability $1 - 0.9^E$. After 20 epochs, $p_{detect} > 0.88$.*

6 Checkpoint Aggregation

At epoch boundaries, validated deltas are merged:

```
1 pub fn merge_deltas(deltas: &[(Delta, DeltaScore)])
2   -> Delta {
3     let total_weight: f64 = deltas.iter()
4       .map(|(_, s)| s.composite as f64)
5       .sum();
6
7     let mut merged = Delta::zeros_like(&deltas[0].0);
8
9     for (delta, score) in deltas {
10       let w = score.composite as f64 / total_weight;
11       merged.add_scaled(delta, w);
12     }
13
14     merged
15 }
```

Listing 5: Weighted checkpoint merge

For Byzantine robustness, coordinate-wise median can replace weighted average when >30% of nodes are suspected adversarial.

7 Model Publisher

7.1 Dual Publication

Merged checkpoints are published to two destinations:

```
1 pub struct ModelPublisher {
2   ipfs: IpfsClient,
3   hf: HuggingFaceClient,
4   signing_key: Ed25519SigningKey,
5 }
6
7 impl ModelPublisher {
8   pub async fn publish(&self, checkpoint: &Checkpoint,
9     job: &Job) -> Result<PublishResult, PublishError> {
10     // 1. Serialize checkpoint
11     let bytes = checkpoint.serialize()?;
12
13     // 2. Upload to IPFS
14     let cid = self.ipfs.add(&bytes).await?;
15
16     // 3. Push to HuggingFace
17     let repo = format!("zooai/{}-epoch-{}",
18       job.model_name, job.epoch);
19     self.hf.create_or_update(&repo, &bytes).await?;
20
21     // 4. Sign publication record
22     let record = PublishRecord {
23       cid: cid.clone(),
24       hf_repo: repo.clone(),
```

```

25         epoch: job.epoch,
26         quality: checkpoint.aggregate_quality(),
27         timestamp: now(),
28     };
29     let sig = self.signing_key
30         .sign(&record.canonical_bytes());
31
32     // 5. Record on-chain
33     self.chain.record_publication(
34         &record, &sig).await?;
35
36     Ok(PublishResult { cid, hf_repo: repo })
37 }
38 }

```

Listing 6: Model publication

7.2 Cross-Verification

The IPFS CID is included in the HuggingFace model card, and the HuggingFace repo URL is recorded alongside the CID on-chain. Users can verify model integrity by:

1. Downloading from HuggingFace.
2. Computing the content hash.
3. Comparing against the on-chain CID.

If the hashes match, the model is authentic. If not, either HuggingFace or the on-chain record was tampered with, and users should prefer the IPFS-hosted copy (content-addressed, immutable).

8 Interaction with Hanzo Compute Chain

Hanzod interacts with the Hanzo compute chain (chain ID 36900) for three operations:

8.1 Stake Verification

Before admitting a node, hanzod queries the on-chain staking contract:

```

1 pub async fn verify_stake(&self, node_id: &NodeId)
2     -> Result<u64, ChainError> {
3     let stake = self.chain.call(
4         "GYMToken",
5         "stakes",
6         &[node_id.as_address()],
7     ).await?;
8     Ok(stake.as_u64())
9 }

```

Listing 7: On-chain stake verification

8.2 Reward Distribution

After each epoch, hanzod computes per-node rewards and submits a batch transaction:

```
1 pub async fn distribute_rewards(&self,
2   epoch: u32,
3   rewards: &[(NodeId, u64)],
4 ) -> Result<TxHash, ChainError> {
5   let workers: Vec<Address> = rewards.iter()
6     .map(|(n, _)| n.as_address()).collect();
7   let amounts: Vec<U256> = rewards.iter()
8     .map(|(_, a)| U256::from(*a)).collect();
9
10  self.chain.send_tx(
11    "MiningPool",
12    "distributeEpochRewards",
13    &[
14      Token::U256(U256::from(epoch)),
15      Token::Array(workers.into_iter()
16        .map(Token::Address).collect()),
17      Token::Array(amounts.into_iter()
18        .map(Token::U256).collect()),
19    ],
20  ).await
21 }
```

Listing 8: Batch reward distribution

8.3 Slashing

When a node fails validation, hanzod submits a slashing transaction:

```
1 pub async fn slash_node(&self,
2   node: &NodeId,
3   pct: u8,
4   evidence: &[u8],
5 ) -> Result<TxHash, ChainError> {
6   self.chain.send_tx(
7     "GYMToken",
8     "slash",
9     &[
10      Token::Address(node.as_address()),
11      Token::U256(U256::from(pct)),
12      Token::Address(self.address()), // reporter
13      Token::Bytes(evidence.to_vec()),
14    ],
15  ).await
16 }
```

Listing 9: Slashing submission

9 Deployment

9.1 Container Configuration

```
1 services:
2   hanzod:
3     image: ghcr.io/zoollabs/hanzod:latest
4     environment:
5       - HANZO_RPC=https://rpc.hanzo.network
6       - HANZO_CHAIN_ID=36900
7       - IPFS_URL=http://ipfs:5001
8       - VALKEY_URL=redis://kv:6379
9       - HF_TOKEN=${HF_TOKEN}
10      - ORCHESTRATOR_KEY=${ORCH_KEY}
11     ports:
12       - "9550:9550" # gRPC
13       - "9551:9551" # metrics
14     depends_on:
15       - kv
16       - ipfs
17
18   kv:
19     image: hanzoai/kv:8
20     volumes:
21       - kv-data:/data
22
23   ipfs:
24     image: ipfs/kubo:latest
25     volumes:
26       - ipfs-data:/data/ipfs
27
28 volumes:
29   kv-data:
30   ipfs-data:
```

Listing 10: Hanzod deployment (compose.yml)

9.2 Kubernetes Deployment

For production, hanzod runs on Zoo's Kubernetes cluster with:

- **Replicas:** 3 (active-passive with leader election via Valkey).
- **Resources:** 4 CPU, 8 GB RAM (orchestrator does not need GPU).
- **Storage:** 100 GB SSD for delta cache.
- **Network:** Exposed via hanzoai/ingress on port 9550.

Metric	Value
Median scheduling latency	340 ms
P99 scheduling latency	1.2 s
Ring construction time (64 nodes)	4.8 s
Ring reconfiguration (1 failure)	8.1 s
Ring reconfiguration (3 failures)	14.3 s
Job completion rate	99.2%
Mean epoch duration	42 min

Table 2: Hanzod scheduling and ring management metrics.

Metric	Value
Deltas validated (50 epochs)	3,200
Mean validation latency	180 ms
Spot-checks performed	312 (9.8%)
Spot-check failures	4 (1.3%)
Slashing events triggered	2
Mean quality score	0.72

Table 3: Delta validation statistics over 50 epochs.

10 Evaluation

10.1 Setup

64-node deployment across 4 regions (US-West, US-East, EU-West, Asia-Pacific). Hardware: 16× A100, 24× RTX 4090, 24× RTX 4080. Model: Qwen3-2507 with QLoRA. Training: 50 epochs on 100K samples.

10.2 Scheduling Performance

10.3 Delta Validation

10.4 Publication

10.5 Orchestrator Failover

We simulated orchestrator crashes during active training:

The 45-second full-node recovery involves leader election among standby orchestrators, state reconstruction from on-chain events, and ring re-establishment.

11 Related Work

Training orchestrators. Ray [1] and DeepSpeed [2] orchestrate distributed training within trusted clusters. Hanzod extends orchestration to untrusted, permissionless networks.

Decentralized compute. Gensyn [3] proposes trustless training verification via probabilistic proofs. Hanzod uses deterministic spot-checking and Ed25519 signatures, which are simpler and sufficient for the Zoo Gym threat model.

Metric	Value
Checkpoints published	50
IPFS upload time (median)	12 s
HuggingFace push time (median)	28 s
On-chain recording time	3 s
Cross-verification pass rate	100%

Table 4: Model publication metrics.

Scenario	Recovery Time	Data Loss
Clean shutdown	0 s	None
Process crash (Valkey intact)	6 s	None
Process + Valkey crash	18 s	Last partial step
Full node failure	45 s	Last partial step

Table 5: Orchestrator failover scenarios.

Job schedulers. SLURM [4] and Kubernetes [5] manage cluster resources but assume trusted infrastructure. Hanzod’s scheduler incorporates stake verification and quality scoring.

Federated learning servers. Flower [6] coordinates federated learning with pluggable aggregation. Hanzod adds ring-topology communication, delta-level validation, and on-chain settlement.

12 Conclusion

Hanzod provides the operational backbone for Zoo Gym: a Rust orchestrator that manages the full lifecycle of decentralized training jobs with 99.2% completion rate and sub-second scheduling latency. By separating job scheduling, ring management, delta validation, and model publication into independent subsystems, hanzod achieves both reliability and auditability. The deterministic state property ensures that any qualified node can reconstruct the orchestrator’s state from on-chain records, eliminating the single point of failure.

Acknowledgments

This work was supported by Zoo Labs Foundation.

References

- [1] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, “Ray: A Distributed Framework for Emerging AI Applications,” *OSDI*, 2018.
- [2] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, “DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters,” *KDD*, 2020.
- [3] Gensyn, “Gensyn Protocol: Trustless Training of Deep Neural Networks,” Whitepaper, 2023.

- [4] A. B. Yoo, M. A. Jette, and M. Grondona, “SLURM: Simple Linux Utility for Resource Management,” *JSSPP*, 2003.
- [5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, 2016.
- [6] D. J. Beutel, T. Tober, D. Stojkovic, S. Joshi, et al., “Flower: A Friendly Federated Learning Framework,” *arXiv:2007.14390*, 2020.
- [7] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning,” *arXiv:1802.05799*, 2018.
- [8] M. Shoeybi et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” *arXiv:1909.08053*, 2019.
- [9] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning,” *arXiv:2402.03300*, 2024.
- [10] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [11] P. Patarasuk and X. Yuan, “Bandwidth Optimal All-Reduce Algorithms,” *J. Parallel Distrib. Comput.*, 2009.
- [12] D. Yin et al., “Byzantine-Robust Distributed Learning,” *ICML*, 2018.
- [13] H. B. McMahan et al., “Communication-Efficient Learning of Deep Networks,” *AISTATS*, 2017.
- [14] J. Benet, “IPFS – Content Addressed, Versioned, P2P File System,” *arXiv:1407.3561*, 2014.
- [15] D. J. Bernstein et al., “High-Speed High-Security Signatures,” *J. Cryptographic Engineering*, 2012.
- [16] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [17] T. Detrmers et al., “QLoRA: Efficient Finetuning of Quantized Language Models,” *NeurIPS*, 2023.
- [18] H. Touvron et al., “LLaMA 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv:2307.09288*, 2023.
- [19] J. Bai et al., “Qwen2.5 Technical Report,” *arXiv:2412.15115*, 2024.
- [20] L. von Werra et al., “TRL: Transformer Reinforcement Learning,” <https://github.com/huggingface/trl>, 2022.
- [21] Opentensor Foundation, “Bittensor: A Peer-to-Peer Intelligence Market,” Whitepaper, 2023.
- [22] V. Buterin, “Ethereum Whitepaper,” 2014.
- [23] Protocol Labs, “Filecoin: A Decentralized Storage Network,” Whitepaper, 2017.
- [24] T. Dao, “FlashAttention-2,” *arXiv:2307.08691*, 2023.
- [25] NVIDIA, “NCCL: The NVIDIA Collective Communications Library,” <https://developer.nvidia.com/nccl>, 2024.